From Instruction Fine-Tuning to QLoRA: A Cost–Quality Study on Financial Instruction Tuning

LLM Fine-Tuning Module · Guided Intermediate Assignment · 100 Marks

Objective

Build a complete parameter-efficient fine-tuning workflow that adapts Qwen2.5-0.5B-Instruct to follow finance instructions from the **Finance Alpaca (gbharti/finance-alpaca)** dataset. You will inspect and prepare the data, build a reusable input pipeline and a hand-written training loop, then fine-tune the same model three ways — full instruction fine-tuning (IFT), LoRA, and QLoRA — comparing two LoRA ranks. You will evaluate answer quality on an untouched test split, plot training loss against validation loss, analyze the errors, and weigh answer quality against trainable parameters, GPU memory usage, and training time.

Resources and Runtime

- Run the supplied starter notebook in Google Colab with a free T4 GPU.
- The notebook installs all Python packages itself (transformers, peft, bitsandbytes, evaluate, bert_score, etc.).
- The public **Finance Alpaca (gbharti/finance-alpaca)** dataset and the Qwen/Qwen2.5-0.5B-Instruct model are downloaded by code.
- A small data subset and short training runs are used deliberately so all three methods finish in one session.
- No API key, account login, Drive mount, or separate file download is required.

Assignment Structure

The notebook is organized into twelve numbered steps. Steps 1–3 (imports & seed, configuration, and helper functions) are provided and carry no marks; the graded steps sum to 100.

Stage	Work	Marks
1	*Imports & seed*	*0 (provided)*
2	*Configuration*	*0 (provided)*
3	*Helper functions (LoRA / QLoRA)*	*0 (provided)*
4	Load dataset + statistics	8
5	Dataset, collate & loaders	14
6	Model load + IFT training	20
7	LoRA (r = 8 and r = 16)	14
8	QLoRA (r = 8 and r = 16)	14
9	Test evaluation (ROUGE-L + BERTScore)	10
10	Plots (train vs validation)	6
11	Error analysis	8

12	Parameter analysis	6
	Total	**100**

What You Must Submit

Submit one completed .ipynb notebook. Every code placeholder and marked reflection must be completed, and all outputs must remain visible. The notebook must run top to bottom after a runtime restart without relying on files outside the notebook.

Assessment Principles

- Reproducibility is graded explicitly: a single seed is set, and the train/validation/test split is fixed before any prompt formatting.
- The prompt must use the tokenizer's chat template, and the loss must be computed on the answer tokens only (completion-only masking).
- Full instruction fine-tuning must be numerically stable — loaded in fp32, since pure fp16 full fine-tuning overflows on this model — and QLoRA must use 4-bit NF4 with fp16 compute on the T4.
- Training configuration must demonstrate warmup, cosine decay, gradient clipping, gradient accumulation, and effective-batch reasoning.
- LoRA and QLoRA must share the same adapter configuration so the comparison isolates backbone precision, and both must be run at ranks 8 and 16.
- Evaluation must use the untouched test split and report both ROUGE-L and BERTScore-F1.
- Written answers must cite the learner's measured metrics, memory and time figures, errors, and diagnostic curves.
- Equivalent correct implementations are accepted; copying hard-coded outputs is not.

Submission Checklist

1. Select a T4 GPU runtime and run all cells top to bottom after a restart.
2. Replace every CODE HERE and Your Answer placeholder, citing your own measured numbers.
3. Check that the loss plots, quality-vs-cost comparison table, quality chart, and error samples are visible.
4. Confirm LoRA and QLoRA both ran at rank 8 and rank 16 and that their results appear in the comparison.
5. Download the executed notebook and submit it without model checkpoints or dataset files.